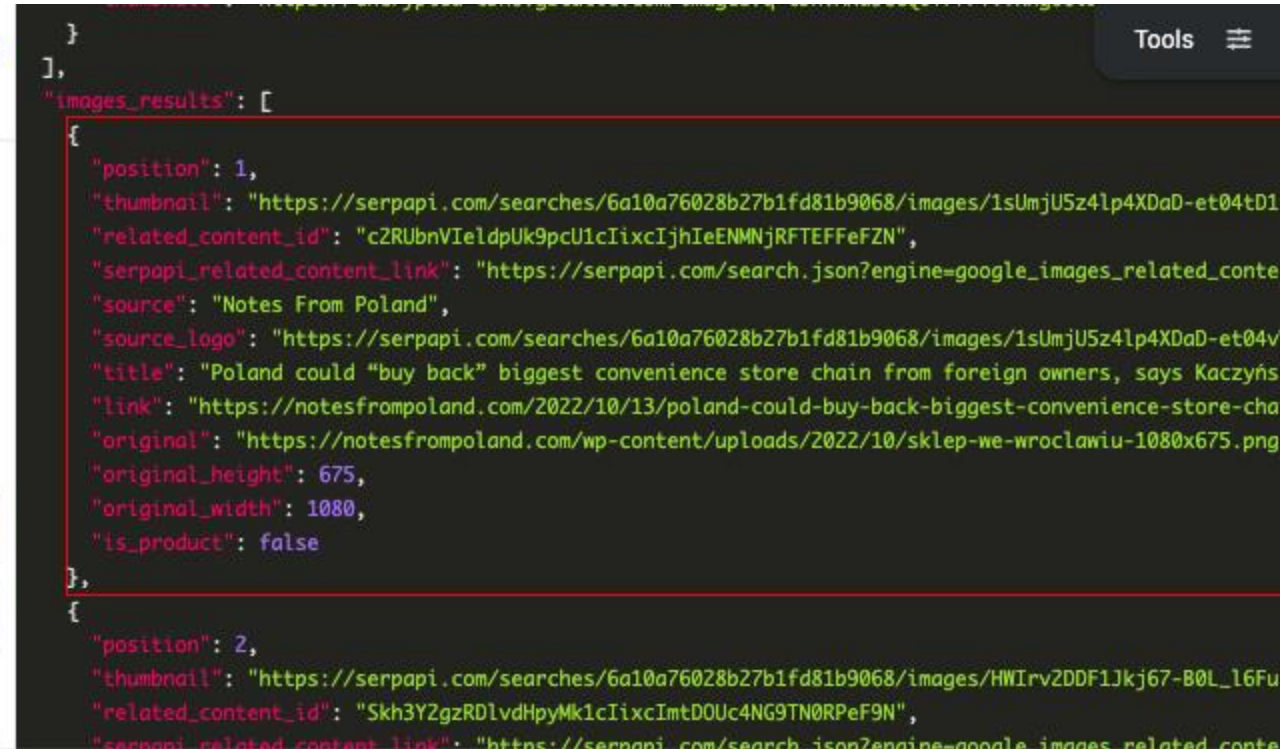
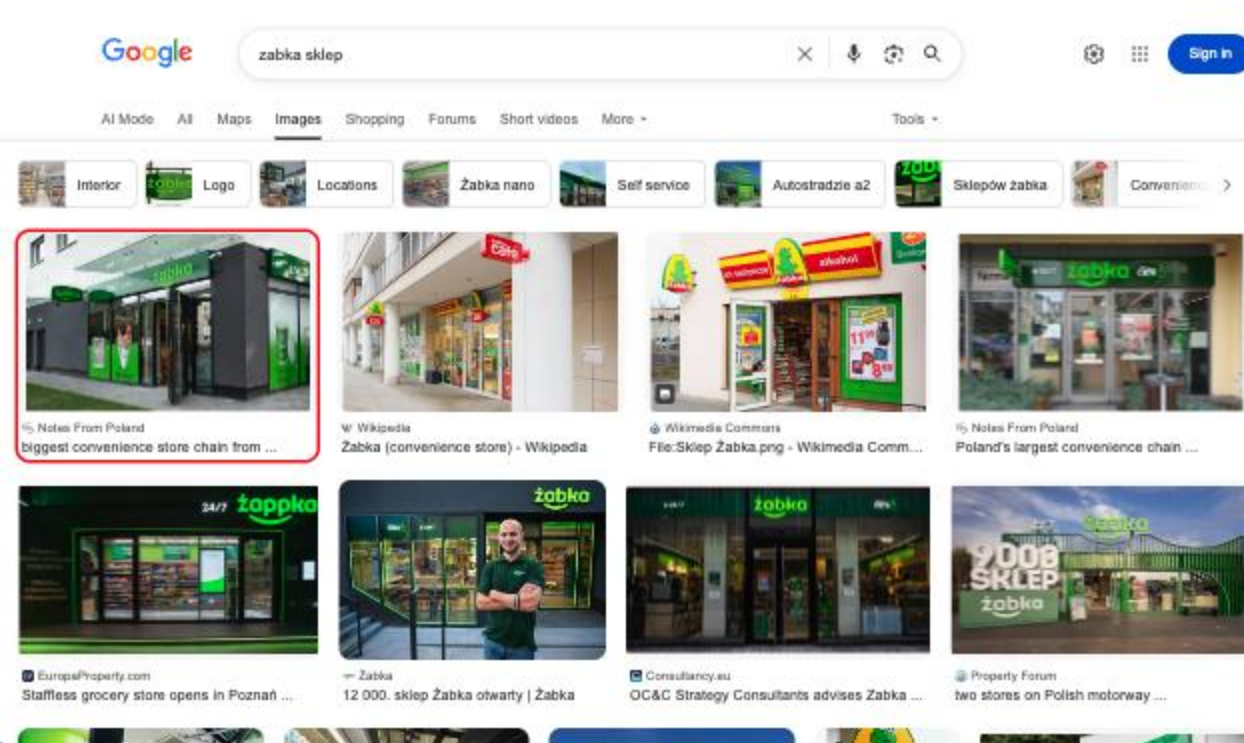


VisitMe

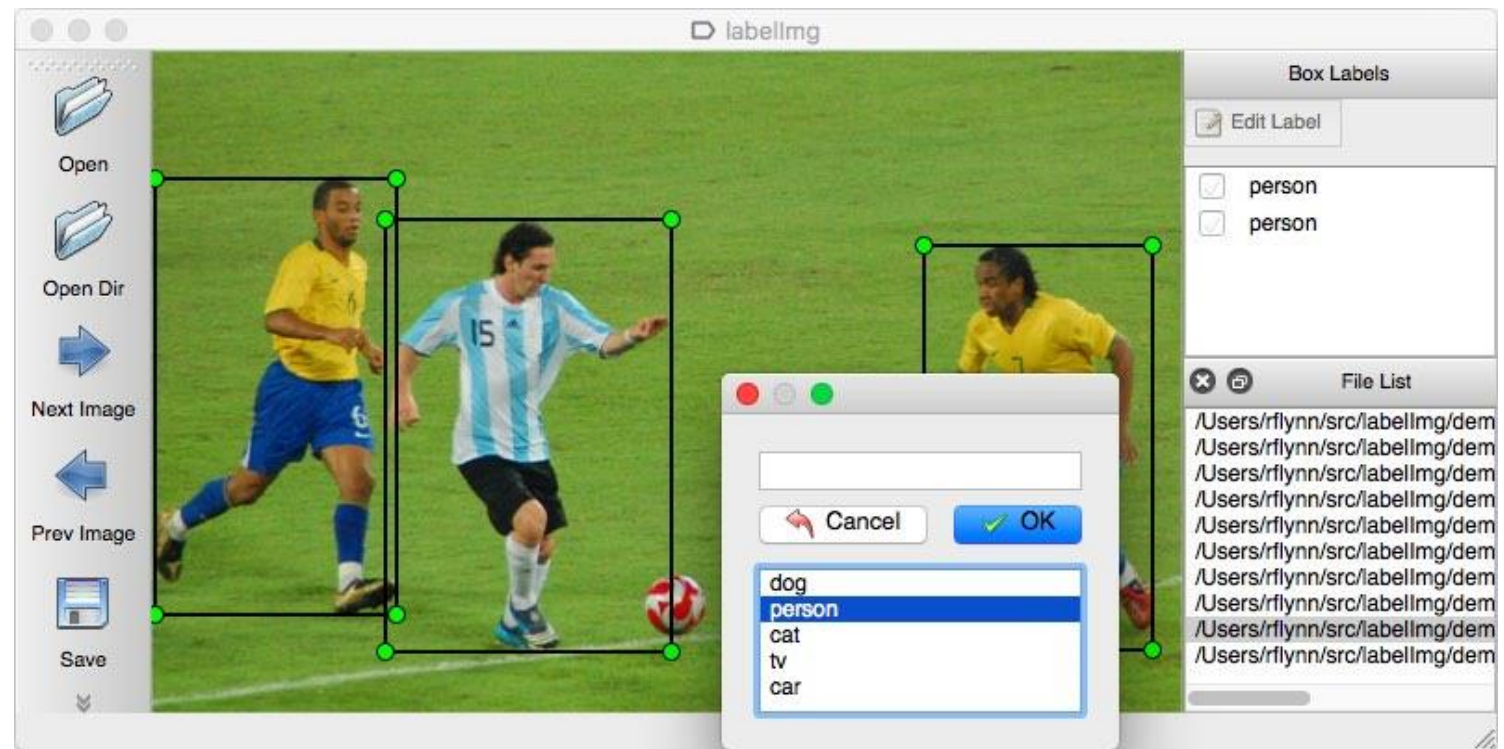
Przygotowanie danych

1. <https://serpapi.com/google-images-api>



Przygotowanie danych

<https://pypi.org/project/labelImg/>



LabelImg is a graphical image annotation tool.
It is written in Python and uses Qt for its graphical interface.
Annotations are saved as XML files in PASCAL VOC format, the format used by [ImageNet](https://image-net.org/).
Besides, it also supports YOLO and CreateML formats.

Przygotowanie danych

0044_Nowa Biedronka na Ursynowie Otwarcie



0007_230 Biedronka Shop Stock Photos Plo



0003_Top 10 Supermarket Retail Chains In



0020_Biedronka otworzyła 23 sklep w Toru



0005_Biedronka z nowym form



(zbiory walidacyjne)

0048_Street View Featuring Zabka Store a



0062_108 m² lokal użytkowy na sprzedaż -



0020_Zabka reaguje na artykuł Pulsu Krak



0090_Zabka Shop in Zakopane Poland Edito



Augmentacja



```
splits = ['train', 'val', 'test']

light_aug = A.Compose([
    A.RandomBrightnessContrast(brightness_limit=0.15, contrast_limit=0.15, p=0.5),
    A.HueSaturationValue(hue_shift_limit=5, sat_shift_limit=20, val_shift_limit=20, p=0.5),
    A.GaussNoise(var_limit=(10.0, 30.0), p=0.3),
    A.Blur(blur_limit=(3, 5), p=0.3),
], bbox_params=A.BboxParams(format='yolo', label_fields=['class_labels']))

medium_aug = A.Compose([
    A.RandomRotate90(p=0.3),
    A.HorizontalFlip(p=0.5),
    A.RandomBrightnessContrast(brightness_limit=0.3, contrast_limit=0.3, p=0.6),
    A.HueSaturationValue(hue_shift_limit=10, sat_shift_limit=30, val_shift_limit=30, p=0.5),
    A.GaussNoise(var_limit=(20.0, 50.0), p=0.4),
    A.Blur(blur_limit=(3, 7), p=0.4),
    A.Downscale(scale_min=0.7, scale_max=0.9, p=0.3),
], bbox_params=A.BboxParams(format='yolo', label_fields=['class_labels']))

augmentations = [light_aug, medium_aug]
```

Przygotowanie danych - statystyki

Dataset	test	train	val	total
biedra/dataset_yolo_biedra	9	51	6	66
biedra/dataset_yolo_biedra_augmented	9	102	12	123
zabka/dataset_yolo	13	91	12	116
zabka/dataset_yolo_augmented	13	182	24	219

Trenowanie YOLO

Modele Żabka:

1. 'AdamW', LR=0.001
 2. 'AdamW', LR=0.002
 3. 'AdamW', LR=0.0016
- +Augmentacja

```
model = YOLO(model_name)
results = model.train(
    data=data_yaml,
    epochs=100,
    imgsz=imgsz,
    batch=batch_size,
    lr0=0.001,
    device=device,
    patience=20, # Early stopping po 20 epokach bez poprawy
    save=True,
    project="runs/train",
    name=experiment_name,
    exist_ok=True,
    pretrained=True,
    optimizer='AdamW',
    verbose=True,
    seed=42,
    plots=True,
)
```

Modele Biedronka:

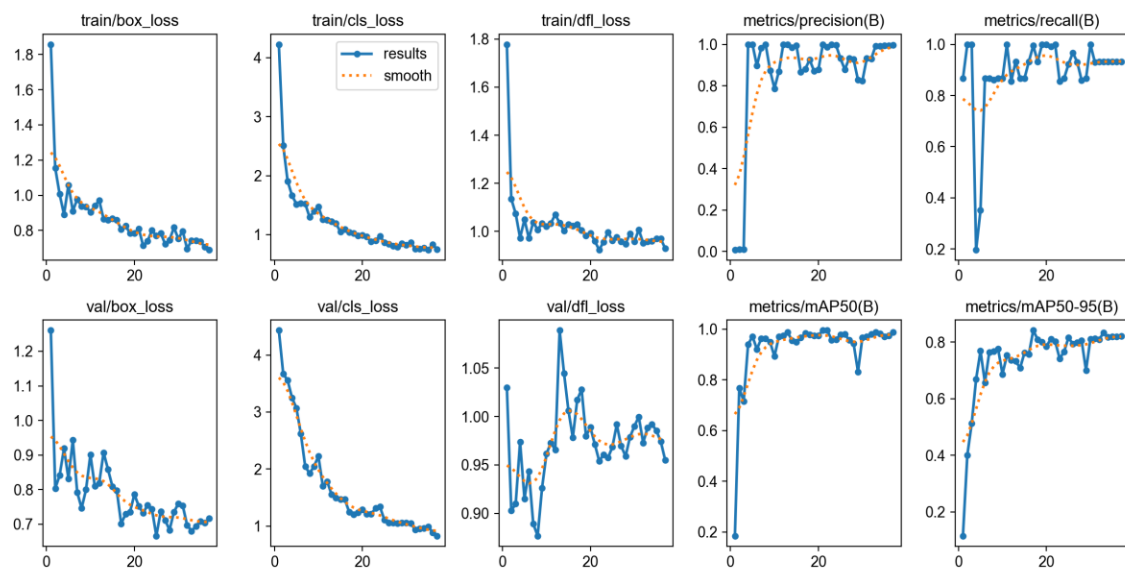
1. 'AdamW', LR=0.001
 2. 'SGD', LR=0.001
 3. 'AdamW', LR=0.001
- +Augmentacja

SGD znajduje **płaskie minima** funkcji straty, podczas gdy AdamW znajduje **ostre minima**.

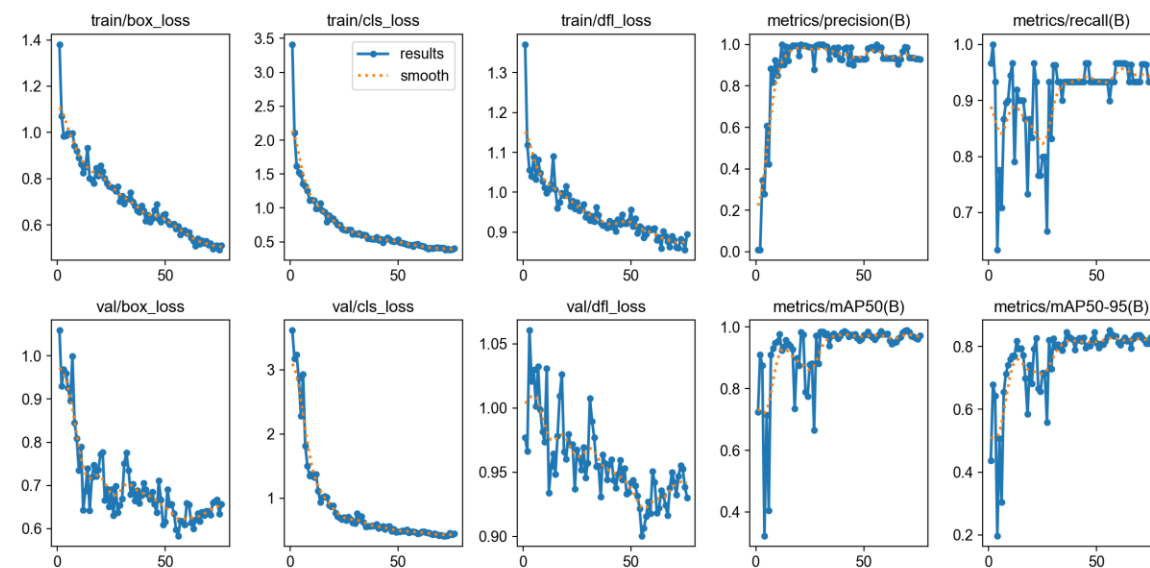
- **Płaskie minimum (SGD)** – małe zmiany wejścia = małe zmiany wyjścia = model **lepiej** radzi sobie z nowymi danymi
- **Ostre minimum (AdamW)** – małe zmiany wejścia = duże zmiany wyjścia = **większe** ryzyko przeuczenia

Trenowanie Yolo - Żabka

'AdamW', LR=0.001



'AdamW', LR=0.001 + Augmentacja

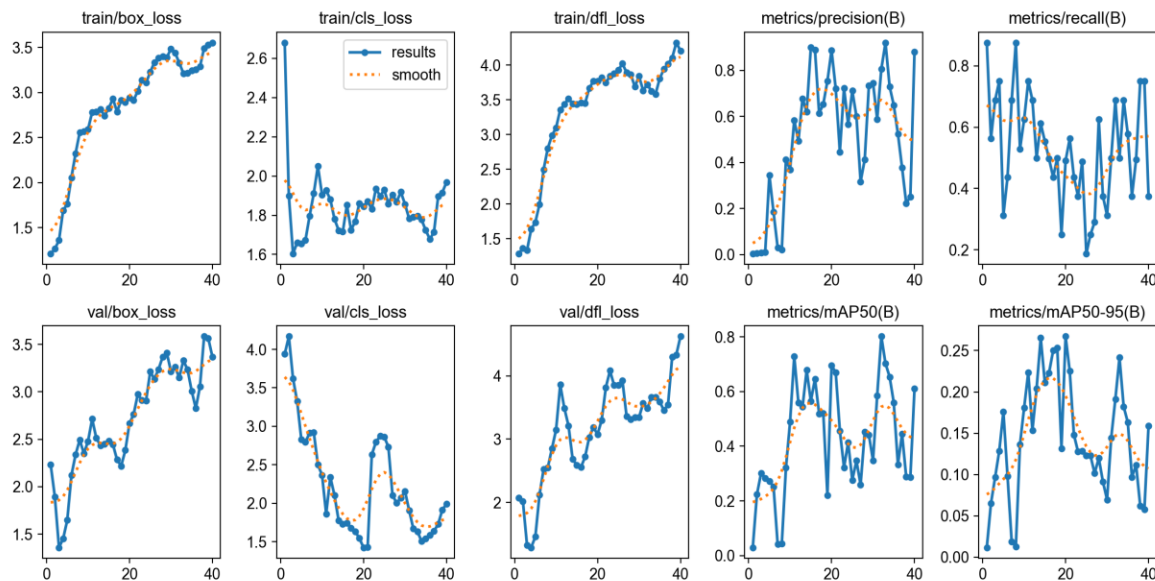


$mAP50(B)$ * Litera (B) -> "Boxes", Liczba 50 oznacza, że model uznaje wykrycie za sukces, jeśli jego ramka pokrywa się z tą prawdziwą w **co najmniej 50%**

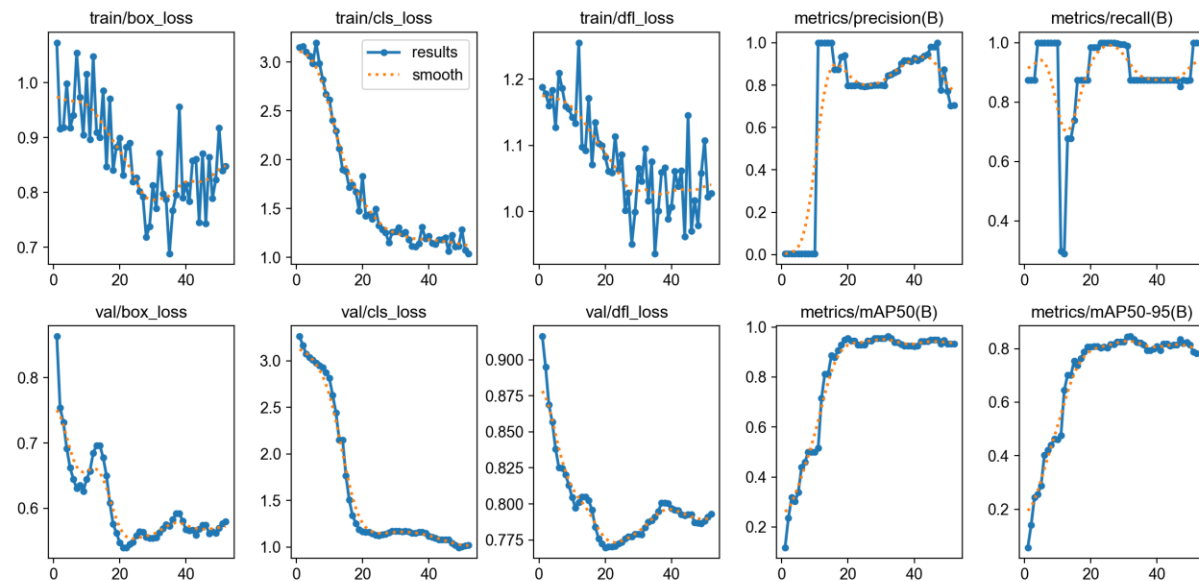
$mAP50-95(B)$ * System oblicza mAP dla progu 50%, potem 55%, 60%... aż do 95% i wyciąga z tego średnią. Pokazuje to, jak idealnie równe są te ramki.

Trenowanie Yolo - Biedronka

AdamW, LR=0.001 + Augmentacja



SGD, LR=0.001



Aplikacja - załadowanie modelu

<https://executorch.ai/> - oficjalny, lekki framework od PyTorch'a stworzony po to, aby uruchamiać modele sztucznej inteligencji bezpośrednio na urządzeniach brzegowych – smartfonach (iOS/Android) i innych urządzeniach.

do naszej aplikacji: <https://docs.swmansion.com/react-native-executorch/>

Aplikacja – pre-processing

Nie możemy załączyć zwykłego pliku .jpg/.heic z telefonu do modelu.

Aparat zwraca piksele w formacie HWC (piksel po pikselu R,G,B,A, R,G,B,A). ExecuTorch i YOLO wymagają formatu **CHW (Channels-First)**. Oznacza to, że w pamięci najpierw muszą leżeć wszystkie czerwone piksele z całego obrazu, potem wszystkie zielone, a na końcu niebieskie.

```
export const imageUriToFloat32CHW = async (
  uri: string,
): Promise<Float32Array> => {
  const resized = await ImageManipulator.manipulateAsync(
    uri,
    [{ resize: { width: MODEL_INPUT_SIZE, height: MODEL_INPUT_SIZE } }],
    {
      format: ImageManipulator.SaveFormat.JPEG,
      compress: 1.0,
      base64: true,
    },
  );

  if (!resized.base64) {
    throw new Error("ImageManipulator nie zwrócił base64");
  }

  const raw = atob(resized.base64);
  const jpegBytes = new Uint8Array(raw.length);
  for (let i = 0; i < raw.length; i++) {
    jpegBytes[i] = raw.charCodeAt(i);
  }

  const decoded = jpeg.decode(jpegBytes.buffer, { useTArray: true });

  const pixelData = decoded.data; // [R,G,B,A, R,G,B,A, ...]
  const pixelCount = MODEL_INPUT_SIZE * MODEL_INPUT_SIZE;

  const tensor = new Float32Array(3 * pixelCount);
  for (let i = 0; i < pixelCount; i++) {
    tensor[i] = pixelData[i * 4 + 0] / 255.0; //r
    tensor[i + pixelCount] = pixelData[i * 4 + 1] / 255.0; //g
    tensor[i + pixelCount * 2] = pixelData[i * 4 + 2] / 255.0; //b
  }

  return tensor;
};
```


KONIEC